Analysis | 1.0.0 |

Problem Description | 1.1.0 |

When designing autonomous systems that require navigation such as warehouse robotics, delivery drones, or self-driving cars, developers are faced with a problem of choosing a suitable pathfinding algorithm. Popular algorithms such as A* excel at traversing a predefined set of vertices and obstacles to find the optimal path in a static environment. However, if the environment changes, it will break. This fails in scenarios where traversing a dynamic environment is required, for instance a self-driving car must continuously adapt to environmental changes. While A* can be re-computed in this scenario, it is not optimal as it wastes computational resources. Or in summary:

There exists no tool to compare between the optimal path and a localized driven algorithm in dynamic environments.

Proposed Solution | 1.1.1 |

This can be solved by modelling a biological archetype that has an innate ability to traverse unknown environments and create a positive feedback loop that allows them to adapt with

minimal changes. The most suitable archetype is an Ant. Because through stochastic movement, it is able to determine a path that is suitable, and if the resource or target is reached, it drops pheromones on the way back allowing other ants to traverse the same path.

To address the problem of choosing between static optimal algorithms and dynamic adaptive algorithms, I will develop a desktop application that allows users to visually compare A* and Ant Colony Optimization algorithm in a procedurally generated, dynamic environment. The application will provide real-time visualization, adjustable parameters, and quantitative metrics to help users show the difference between ACO and an optimal path algorithm

Stakeholders | 1.3.0 |

Primary Stakeholder | 1.3.1 |

Stakeholder	Role	Need	Consequence
Robotics Engineer	Designs autonomous systems	Must choose algorithms suitable for their autonomous creation	Allows to compute path changes in a dynamic environment and implement the said algorithm in their chosen system
Computer Science Teacher / Lecturer	Teaches algorithms at A-Level or university	Visual representation of the algorithms to demonstrate how algorithms differ	Control parameters that directly affect the visuals, therefore allowing the concept to be easier to explain
A-Level / Undergraduate Student	Learning pathfinding algorithms	To comprehend the differences between optimal path and ACO	Ability to experiment and see results

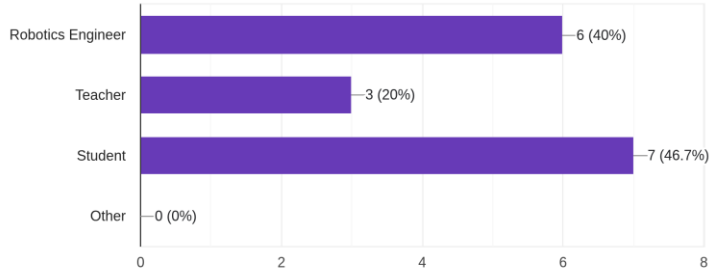
Secondary Stakeholder | 1.3.2 |

Stakeholder	Role	Need	Consequence
Game Developer	Design NPC's, Enemies	Create natural pathways for entities	Creates natural looking pathways that doesn't have much overhead
Hobbyist / Enthusiast	An interest in how biological entities are used to model path algorithms	Simply wants to experiment	Allows an ease of understanding and how different metrics compare
Researcher	Studies biological entities	A need to visualize how different metric causes entities to behave differently	Ability to craft reports out of the controlled metrics

Questionnaire | 1.4.0 |

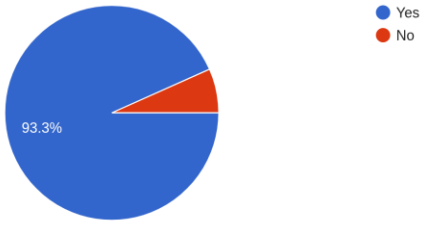
Choose an appropriate role

15 responses



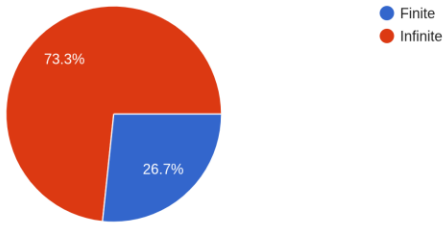
Real Time Visualization

15 responses



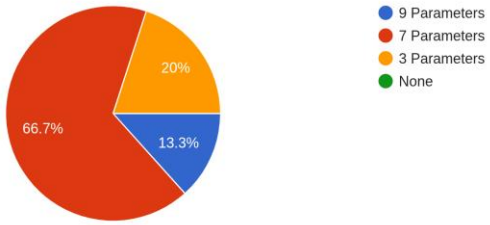
Grid Size

15 responses



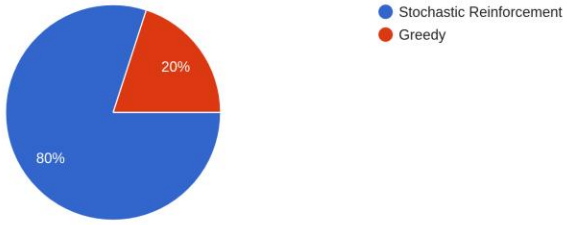
Real Time Parameters

15 responses



Rule for the main path-finding algorithm

15 responses



Summary | 1.4.2 |

Aspect	User Preference / Finding	Design Decision
Target Audience	Engineers, Students, Teachers	Explain concepts with mathematical rigor to show foreign concepts that they may not have seen before
Interface Type	93.3% wants it to be a visual implementation	Develop a custom interface that shows entity movement + emergent trails
World Size	73.3% voted for an infinite world	Implement a chunk-based scrolling world with an observer
Tunable Parameters	66.7% of users want 7 parameters	Create a user interface with 7 parameters + sliders to adjust
Movement Rule	80% of users wants a stochastic implementation of pathfinding	Implement experimental entity to move probabilistically with tuned parameters that control behaviors

Current Systems | 1.5.0 |

System / Approach	Type	Strengths	Weaknesses	Relevance to Proposed Solution
A* Pathfinding	Traverse a static environment	optimal path (with heuristics)	Needs global knowledge + works in static environment otherwise needs to be recalculated	A comparison metric to my implantation of pathfinding
Dijkstra's Algorithm	Static optimal shortest path	Finds a guaranteed optimal path	Slower than A*	Importance of heuristic optimization
Ant Colony Optimization (ACO)	Adaptive metaheuristic	Doesn't need global knowledge	Slower convergence, isn't always optimal	Core algorithm for the app

Archetype Evaluation | 1.6.0 |

Ant Colony Description | 1.6.1 |

Ant colonies are amongst the largest cohesive communities. Despite having a limited spatial perception, it manages to communicate effectively via chemicals with external stimuli not affecting the trail formed by the predecessor. Their foraging systems coordinate hundreds of thousands of individuals that lead towards the same repository. This causes it to be an excellent specimen to simulate stochastic behavior to create an optimized algorithm for a dynamically changing environment.

Research | 1.6.2 |

Goss et al. (1989) | '*Self-organized shortcuts in the Argentine ant***'**

The question posed is as follows, 'How can an animal with only limited and local navigational information achieve this?'. Naturally referring to how ants can move between different points regularly. A summary of the experiment is as follows:

1. Laboratory colonies are given access to a food source in an arena linked to the nest by a bridge.

2. So, a forager may choose either of the 30-degree slopes to reduce bias or to "minimize perturbation" - Do note both path leads to food, while one takes longer to reach.

Results | '*Self-organized shortcuts in the Argentine ant***'**

After a short period of observation, the researchers observed that the ant converged towards the shortest path to the food repository. This is startling because ants, as previously stated, have a very limited perception range, so how can they determine which path to choose from? This can be answered by either tactile communication or pheromones. But the researcher removed the pheromones and then used the same object to retest. This caused the initial probabilistic

movement to occur again, therefore, strongly inferred that pheromones is the main driving force of positive-feedback reinforcement loop. Not tactile communication.

More On Ants | '*Trail laying behavior during food recruitment in ants'*

An interesting behavior found in this article is that forager ants do not initially lay pheromones along the path, but instead in the path of return when they find a suitable resource for the colony. Further investigation suggests that the ants remember some simple landmarks or some nests have a particular chemical which allows them to navigate back. Another interesting aspect is that the ants have a set radius that is correlated with the energy that the ant has and therefore initiating a return behavior.

Initial Modelling | 1.7.0 |

Divergence | 1.7.1 |

Divergence metric may be defined as the normalized difference between the difference of the emergent path at iteration t and the optimal path.

$$D_t = \frac{L_E(t) - L_O}{L_O}$$

Terrain Resistance | 1.7.2 |

Let the continuous spatial domain be

$$\Omega \subset R^2$$

Terrain resistance may be defined as the continuous noise spectrum that resides in the spatial domain of the environment. Where a noise is defined as

$$N : \mathbb{R}^2 \rightarrow [-1 , 1]$$

Therefore, defining terrain resistance to be

$$\tau(x , y)$$

Consider the normalized noise belonging to $[0 , 1]$ which therefore must be translated into the interval of $[\tau_{min} , \tau_{max}]$

$$\tau(x\,,\,y)\;=\;\tau_{min}\;+\;\left(\frac{N(x\,,\,y)+1}{2}\right)(\tau_{max}\;-\;\tau_{min})$$

Cost Definition | 1.7.3 |

A cost may be defined as the multiple of the distance and the terrain weight. The distance from vertex i and vertex j may be defined as

$$S_{ij}\;=\;\sqrt{(x_i\;-\;x_j)^2\;+\;(y_i\;-\;y_j)^2}$$

Therefore, defining cost to be

$$C_{ij}\;=\;\tau_{ij}S_{ij}$$

Energy Definition | 1.7.4 |

Energy Λ may be defined as a local constant for an ant that depletes linearly with respect to the product of terrain resistance and the distance travelled between nodes i and j. Therefore, representing energy as

$$E_{ant}(t\;+\;1)\;=\;E_{ant}(t)\;-\;C_{ij}$$

Emergent Pathway Definition | 1.7.5 |

Total length obtained iteratively from selecting the current vertex and the adjacent vertex with the greatest pheromone weight, $W(x_j\,,\,y_j)$ chosen over t iterations to the resource vertex. If vertices have a maximum W of equivalent value, then a vertex will be chosen as

$$v\;\sim\;Uniform(v_i\,,\,v_j)$$

Optimal Pathway Definition | 1.7.6 |

Let a path be defined as $p\;=\;(S,\,v1\,,\,v2\,,\,\ldots\,,\,R)$

Let the total cost of a path be $C(p)\;=\;C_{(s\,,\,v_1)}\;+\;C_{(v_1\,,\,v_2)}+\cdots+\;C_{(v_k\,,\,R)}$

Therefore, the optimal path must be defined as

$$L_O = \min_{p \in P} C(p)$$

Where P is the set of all paths from S to R.

Modelling Pheromone Decay rate | 1.7.7 |

Chemical evaporation decay rate is defined as

$$\frac{dA}{A} = -k\,dt \text{ Or simply... } \frac{dA}{dt} = -kA$$

Let ξ_{ij} represent intensity of pheromone at edge $(i\,,\,j)$ and denote β to represent the constant for the evaporation of pheromone strength.

Denote a function where the number of pheromones deposited is dependent on the number of ants at edge E_{ij}

$$f(t) = \varepsilon\sigma_{ij}(t)$$

Where $\sigma_{ij}(t) =$ number of ants returning using the edge $(i\,,\,j)$ at iteration t . Where ε denotes the number of pheromones deposited per ant traversal.

$$\ldots \frac{d\xi_{ij}}{dt} = \varepsilon\sigma_{ij}(t) - \beta\xi_{ij} \ldots$$

To make it algorithmically suitable, make the continuous derivative into a discrete / iterative function by using Eulers forward difference approximation

$$\frac{dx}{dt} \approx \frac{x(t+1) - x(t)}{\Delta t}$$

$$\frac{\xi_{ij}(t+1) - \xi_{ij}(t)}{\Delta t} = \varepsilon\sigma_{ij}(t) - \beta\xi_{ij}(t)$$

$$\xi_{ij}(t+1) = (1 - \beta\Delta t)\xi_{ij}(t) + \varepsilon\Delta t\sigma_{ij}(t)$$

$$\Delta t \approx 1$$

$$\boxed{\xi_{ij}(t+1) = (1-\beta)\xi_{ij}(t) + \varepsilon\sigma_{ij}(t)}$$

Probability Driven Movement | 1.7.8 |

Let vertex i have n neighbors with adjacent vertices indexed by $j = \{1\,,\,2\,,\,3\,,\,\ldots\,,\,n\}$

The probability of an ant choosing a neighbor with a fixed current index i and neighbor index j will be represented as $P_{ij}(t)$.

Assume Pheromone strength $\xi_{ij}\ (t)$ is proportional to $P_{ij}(t)$.

$$P_{ij}(t)\ \alpha\ \xi_{ij}(t)$$

Introduce a constant h to form an equation such that

$$P_{ij}(t) = \ h\xi_{ij}\ (t)$$

The sum of all probability from $P_{ij}(t)$to $P_{ik}(t)$ must equal to 1 such that

$$\sum_{k\ =\ 1}^{n} P_{ik}(t)\ \ =\ 1$$

So, consider the sum of both sides, which therefore introduces the alternate form of h

$$h\ \ =\ \frac{1}{\sum_{k\ =\ 1}^{n} \xi_{ik}(t)}$$

In consideration of $\sum_{k\ =\ 1}^{n} \xi_{ik}(t) = \ 0$ introduce a new baseline constant ξ_0 that accounts for it. Therefore, create an augmented field $\sim\ \xi_{ij}(t) = \ \xi_{ij}\ +\ \xi_0$

Substituting the augmented field back into the probabilistic proportionality yields

$$P_{ij}(t)\ \ =\ \frac{\sim \xi_{ij}(t)}{\sum_{k\ =\ 1}^{n} \ \ \sim \xi_{ik}(t)}$$

Return Movement | 1.7.9 |

A path may be defined as $p\ =\ \{S\ ,\ v1\ ,\ v2\ \ldots\ ,\ R\}$. P will be defined such that it's the set of all paths possible to the Resource Vertex. Therefore $p\ \in\ P$ .To return the path will simply be the reverse of chosen path p or simply

$$Rev(p)$$

Objectives | 1.8.0 |

World Generation | 1.8.1 |

Ref	Description
1.7.1.1	Create an infinite world via chunking
1.7.1.2	Generate terrain using Perlin Noise
1.7.1.3	Use the defined terrain resistance
1.7.1.4	Place obstacles for the ants
1.7.1.5	Create nodes at set intervals
1.7.1.6	Create an observer for the world

Ant Logic | 1.8.2 |

Ref	Description
1.7.2.1	Create different Ant entities
1.7.2.2	Implement energy system: $E_{ant}\,(t+1) = E_{ant}\,(t) - C_{ij}$
1.7.2.3	Reset ants when energy exhausted or after successful return
1.7.2.4	Ant should have a localized spatial perception
1.7.2.5	Ants should remember the path they came from

Movement Rules | 1.8.3 |

Ref	Description
1.7.3.1	Implement probabilistic movement based on pheromone levels.
1.7.3.2	Calculate edge cost as $C_{ij} = \tau_{ij} S_{ij}$
1.7.3.3	Ant won't move when it runs out of energy
1.7.3.4	Handle cases with no valid neighbors
1.7.3.5	Return journey is incurred when ant finds a resource point

Pheromone System | 1.8.4 |

Ref	Description
1.7.4.1	Model pheromone decay as $\xi_{ij}(t+1) = (1-\beta)\xi_{ij}(t) + \varepsilon\sigma_{ij}(t)$
1.7.4.2	Deposit pheromones only on return journey
1.7.4.3	Reward the shorter path more - Reinforcement
1.7.4.4	Clear pheromones if its below energy
1.7.4.5	Add baseline pheromone ξ_0 to all edges

Visualization | 1.8.5 |

Ref	Description
1.7.5.1	Display ant state with colors
1.7.5.2	Draw pheromone trail via intensity control - orange
1.7.5.3	Show greedy path - cyan
1.7.5.4	Render objects belonging to world
1.7.5.5	Add visual aspects to make the environment pleasant to look at

User Interface | 1.8.6 |

Ref	Description
1.7.6.1	Provide real-time parameter adjustment sliders for β, ε, ξ_0, α, ant count, energy and speed
1.7.6.2	Allows the pheromones to be reset - button
1.7.6.3	Display Statistics of current iteration
1.7.6.4	Allow emergent or optimal path to be hidden on/off
1.7.6.5	Control simulation speed
1.7.6.6	Enable camera movement with arrow keys

Comparison Metrics | 1.8.7 |

Ref	Description
1.7.7.1	Calculate emergent path length $L_E(t)$
1.7.7.2	Display divergence
1.7.7.3	Implement A* algorithm for optimal path
1.7.7.4	Calculate optimal path length L_O using terrain-weighted costs

1.7.7.5	Compute and display divergence $D_t = \frac{L_E(t) - L_O}{L_O}$
1.7.7.6	Allow toggling between the optimal path and ACO

Design | 2.0.0 |

Overview | 2.1.0 |

The program will be implemented in Python, where pygame will be used to create the infinite world where the entities will live. While the UI will be created by customtkinter. Where code will be written in a modular manner, via OOP. This will be done so that systems can avoid being tightly coupled and therefore easily testable, and as an added effect, allow new code to be implemented with ease – Disregarding the complexity of the algorithm itself.

This Design section will discuss aspects of code quality, modularization, class interaction, algorithmic design, inheritance, or any other OOP concepts.

System Architecture | 2.2.0 |

Code Quality | 2.2.1 |

The system must follow these set of rules

Encapsulation | 2.2.2 |

Private Attribute data

The individual classes must have a set of internal data that can **never** be accessed unless explicitly stated otherwise, which is simply having getters and setters in the appropriate places. This will prevent tightly coupled systems from occurring and therefore allow developers to test individual classes with ease.

Foremost consideration, there must be a Main class that sees all the other classes. This class will act as a massive state manager that controls the inputs and outputs of all the other classes. This in return allows data from classes to be passed into the necessary place without directly referencing or creating an instance of another class in the said class and thereby maintaining individual integrity.

Below shows an abstraction of python code that demonstrates Private attribute data, getters and setters ...

Encapsulation

```
def init(self, node_position, max_energy=5000.0):
      self._returning = False
      self._energy = max_energy

@property
def energy(self):
   return self._energy

@property
def returning(self):
   return self._returning
```

This clear demonstrates that the protected attributes are never meant to be accessed directly but in a similar fashion to the abstraction above. This is so that private data of that class won't be mutated un-intentionally and cause a different - hard to trace bug to occur. Therefore, in my implementation, I will use the single underscore to show it's a protected attribute and use clear @property and @x.setter to show what can be accessed and changed.

And for the main class exemplar

State Manager

```
class Main:
      def __init__(self , ...):
              self.World = World()
              self.Generation = ()
              self.Render = Render(Self.Generation.GetPosition)
```

As you can see in the demonstrated code (abstracted), sub classes must never be able to see each other,

unless they are related, then they can be related by inheritance, otherwise it's best to avoid doing such a thing. Since it's better to simply create a Main class that holds the instance of the classes and pass in parameters when needed. And furthermore, if an instance of generation was indeed made in render, then it could cause circular dependencies to occur and break the code. This in addition keeps different classes loosely coupled and therefore allows individual classes to be tested appropriately.

Managing Constants

Data constants must be clearly identified in the code base without ambiguity. This includes the derivation of the said constant in the documentation – unless it's typical arithmetic operations such as division by 2, which in this case is not necessary to do so. The creation of constants can be in the classes' constructor, but it is advised to have class specific constants stored in its own dataclass to have a centralized control panel for the developer to tweak if necessary.

As demonstrated by the abstracted code:

Data Collection

```
@dataclass
class Colour:
      background_grey: tuple[int, int, int] = ...
      green: tuple[int, int, int] = ...
      yellow: tuple[int, int, int] = ...
```

This abstraction demonstrates how related constants can be stored and therefore allows a central ‘dictionary’ which holds the record of the constants clearly.

Modularization

Repeated code must be cleaned up and made into a subroutine, If possible. This allows developers to effectively debug necessary sections without a major overhaul of hundreds of lines of code. And it simply looks cleaner and makes use of time efficiently.

For instance, say you are using the square root method multiple time, you can instead do something similar like the abstracted python code below

Function Exemplar

```
def SquareRoot(n):
        return n ** 0.5

Or simply...
Square = lambda n: n** 0.5
```

Where the first example should be used if square root is done throughout the entire code, or otherwise a local function which is the second example to populate the internals of a specific subroutine.

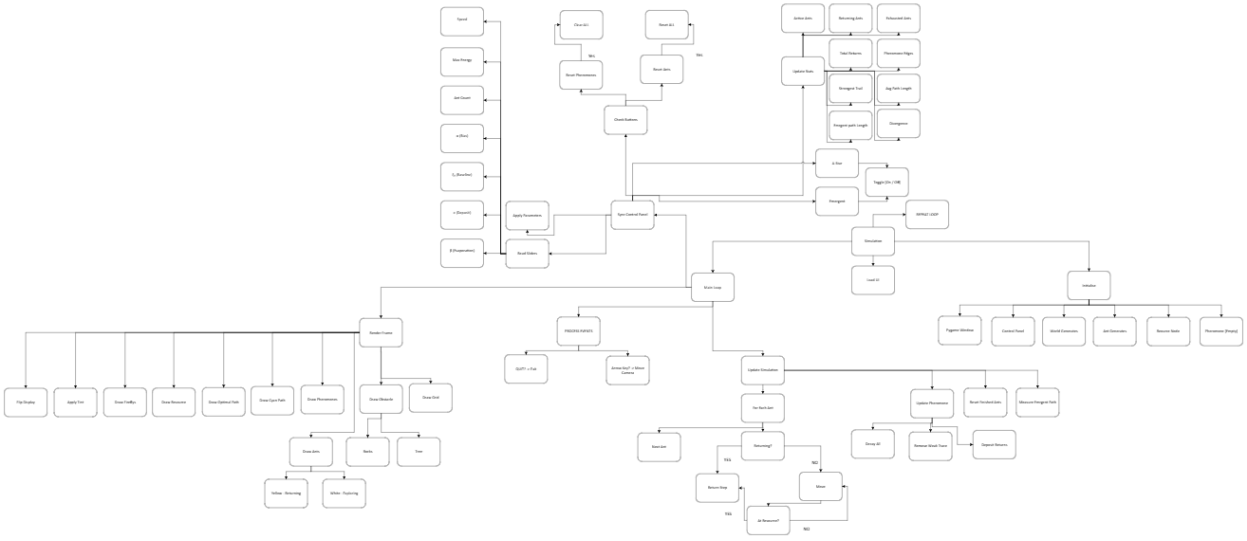
Defensive Programming

Predict potential errors that can occur while the program runs and use python exceptions to handle it or predict the behavior which will occur in result of this – although not implemented in a program, in my analysis I stated that the augmented pheromone field $\sim \xi_{ij} = \xi_{ij} + \varepsilon$. Why? Because otherwise $P_{ij}(t) = \frac{\sim \xi_{ij}(t)}{\Sigma_{k=1}^{n} \sim \xi_{ij}(t)}$ will cause a zero-division error in a program and is mathematically invalid (unless you are modelling a black hole??).

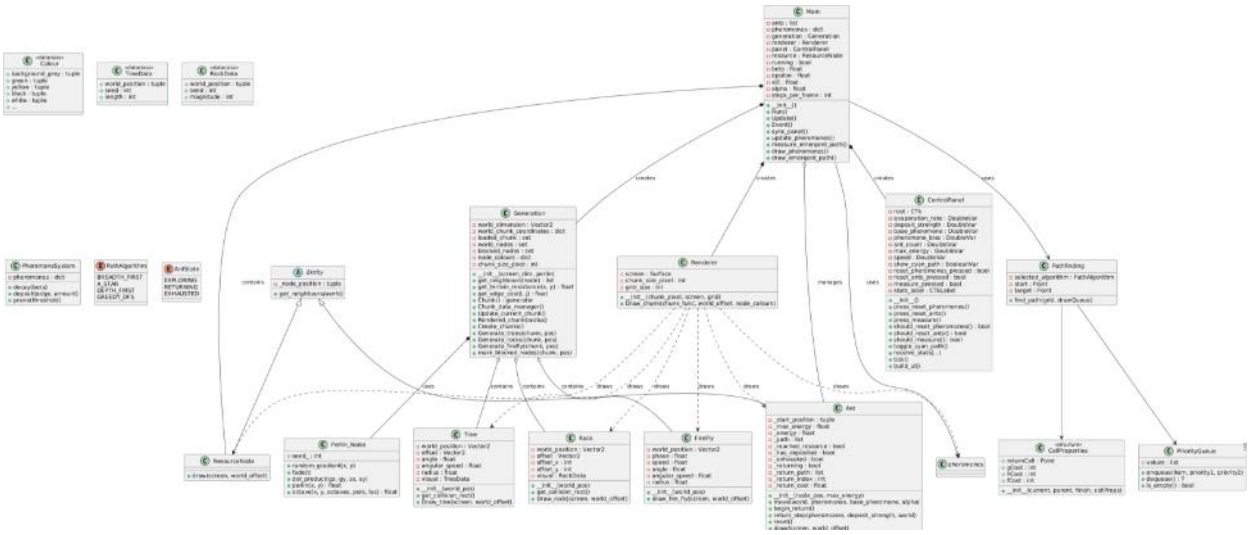
Code Clarity

Use type hints at the appropriate place and comments explaining deeply nested logic, or of a complex structure and ideas. This in consequence allows other people to understand your code and debug/add new features. Instances of type hinting have been used in the above abstractions. An excessive number of comments is always better than no comments.

Program Flow | 2.2.3 |



Class Diagram | 2.2.4 |



IPSO Chart | 2.2.5 |

Input	Process	Storage	Output
Launch application	Initializes Pygame window, customtkinter control panel, world generation (chunks, Perlin noise), ant list, resource node, pheromone dictionary, default parameters.	Main, Generation, ControlPanel, ant list, pheromone dictionary, resource node	Pygame window displayed, control panel shown, initial world rendered.
Evaporation rate slider (β) changed	Update self.beta in main	Self.beta in main	Varies in pheromone decay rate
Deposit strength slider (ε) changed	Update self.epsilon in main	Self.epsilon in main	Varies pheromone deposit
Base pheromone slider (ξ_0) changed	Update self.ix0 in main	Self.xi0 in main	Varies baseline pheromone

Pheromone bias slider (α) changed	Update self.alpha in main	Self.alpha in main	Varies ants' sensitivity to pheromone
Ant count slider changed	Update number of ants in existence	Self.ants in main	Varies number of ants in simulation
Max energy slider changed	Update the max energy in all ants	Sellf.max_energy in main	Varies energy of ants
Speed slider changed	Updates the speed of ants	Self.steps_per_frame in main	Iterations per frame
"Reset Pheromones" button clicked	Clears pheromone memory	pheromones dict, successful_returns (int), recent_path_lengths list.	Pheromones disappear from the screen
"Reset Ants" button clicked	Resets to initial state	Every variable in ants internal	All ants respawn at the start, other metrics still exists
"Show Cyan Path" checkbox toggled	Toggle to show emergent path	show_emergent_path bool.	Cyan path disappears/shows
"Show A* Path" checkbox toggled	Toggle to show A* path (optimal path)	show_astar_path bool; astar_path	Magenta path disappears/shows
Arrow keys pressed	Scrolls through the world	world_dimension Vector2 in Main.	Camera moves
Close window / QUIT event	Closes the program	None.	Application closes.

Pseudo Code / Architecture | 2.3.0 |

Infinite World | 2.3.1 |

The goal is to create a scrollable world that renders the current and the adjacent chunk and unloads the traversed chunks outside the adjacent chunks to decrease latency.

Chunk Approximation

The unbounded spatial domain $\Omega \subset R^2$ will be partitioned into discrete chunks where indexed by integer coordinates$(i\,,\,j) \in \mathbb{Z}^2$, where a chunk covers a rectangular domain $\in \Omega$

Define a chunk to have a spatial dimension of

$$\dots\ (w\Delta x\,,\,h\Delta y)\ \dots$$

Let a chunk be defined as such:

$$C_{ij} = [iw\Delta x\,,\,(i+1)w\Delta x) \cdot [jh\Delta y\,,\,(j+1)h\Delta y]$$

And therefore, defining the partition of the domain:

$$\Omega = \bigcup_{(i\,,\,j)\,\in\,\mathbb{Z}^2} C_{ij}$$

Where $C_{IJ} \cap C_{kl} = \emptyset$ for $(i\ ,\ j) \neq (k\ ,\ l)$. Therefore, allowing world x and world y to be converted by the process of mapping such that:

$$\varphi : \Omega \to \mathbb{Z}^2$$

$$\varphi(x\ ,\ y) = \left(\frac{x}{w\Delta x},\ \frac{y}{h\Delta y}\right)$$

The following Pseudo-code describes process of defining an unbounded world that dynamically renders adjacent chunk, where the chunk is defined in the preceding section. Noise implementation is deliberately abstracted because it's not needed to create a scrolling map but is referenced because the world entities that resides in the world is instantiated here **which it doesn't have to be for this implementation to be true.**

Block One

```
SUBROUTINE UpdateWorld(playerPosition)

        currentRegion ← DetermineRegion(playerPosition)
        visibleRegions ← GetNeighbouringRegions(currentRegion)

        FOR EACH region IN visibleRegions DO
                IF region is not initialised THEN
                        Initialise region
                END IF
                Generate terrain for region
                Place static entities in region

        END FOR
        Render visibleRegions relative to playerPosition

END SUBROUTINE
```

Block Two

```
SUBROUTINE Initialise(region)

        Create empty terrain data
        Create empty entity list

 END SUBROUTINE
```

Block Three

```
SUBROUTINE Generate terrain for region

        Sample environmental variation across region
        Assign terrain types based on sampled values

END SUBROUTINE
```

Perlin Noise | 2.3.2 |

Perlin Noise algorithm produces outputs in the form of 'noise' that is random but continuous in nature. Thereby, it can be used to model terrain algorithmically to provide a natural looking terrain. It can be modelled generally as such

$$N(x\,,\,y)\;=\;(1\;-\;v\,')n_{x0}\;+\;v\,'n_{x1}$$

Where $n_{x0}$is the linear interpolation along one of the x axis and $n_{x1}$is the adjacent axis to x which is also interpolated, where v is the offsets that is fed into the fade function of

$$f(t)\;=\;6t^5\;-\;15t^4\;+\;10t^3$$

Perlin Main

```
SUBROUTINE PerlinNoise(x, y)
    cellX ← FLOOR(x)
    cellY ← FLOOR(y)

    localX ← x - cellX
    localY ← y - cellY

    grad00 ← RandomGradient(cellX, cellY)
    grad10 ← RandomGradient(cellX + 1, cellY)
    grad01 ← RandomGradient(cellX, cellY + 1)
    grad11 ← RandomGradient(cellX + 1, cellY + 1)

    dot00 ← DotProduct(grad00, localX, localY)
    dot10 ← DotProduct(grad10, localX - 1, localY)
    dot01 ← DotProduct(grad01, localX, localY - 1)
    dot11 ← DotProduct(grad11, localX - 1, localY - 1)

    smoothX ← Fade(localX)
    smoothY ← Fade(localY)

    interpX0 ← Lerp(dot00, dot10, smoothX)
    interpX1 ← Lerp(dot01, dot11, smoothX)
    result ← Lerp(interpX0, interpX1, smoothY)

    RETURN result
END SUBROUTINE
```

Perlin Helper

```
SUBROUTINE Fade(t)
    RETURN t * t * t * (t * (t * 6 - 15) + 10)
END SUBROUTINE


FUNCTION Lerp(a, b, t)
    RETURN a + t * (b - a)
END SUBROUTINE


SUBROUTINE RandomGradient(x, y)
    angle ← HASH(x, y) * 2 * PI
    RETURN (COS(angle), SIN(angle))
END SUBROUTINE
```

The current abstraction allows the creation of one instance of Perlin. This does create smooth looking terrain or noise, but it looks too smooth. So, the solution is to layer multiple perlins on top of one another with different amplitude and frequency to mimic superposition and therefore make it look noisier but still keeping it continuous. This is demonstrative by the summation below

$$FBM(x\,,\,y)\;=\;\Sigma_{k\,=\,0}^{n\,-\,1}\,p^{k}\cdot N(L^{k}x\,,\,L^{k}y)$$

And later normalized to a range of $[-1\,,\;1]$ by dividing the resultant by the total amplitude.

Perlin Summation

```
SUBROUTINE FBM(x, y, octaves, persistence, lacunarity)

    total ← 0
    amplitude ← 1
    frequency ← 1
    maxAmplitude ← 0

    FOR i FROM 1 TO octaves DO
        total ← total + PerlinNoise(x * frequency, y * frequency) * amplitude
        maxAmplitude ← maxAmplitude + amplitude
        amplitude ← amplitude * persistence
        frequency ← frequency * lacunarity
    END FOR

    RETURN total / maxAmplitude
END SUBROUTINE
```

Modelling terrain resistance via Perlin | 2.3.3 |

Now as previously defined in analysis terrain resistance is defined to be

$$\tau(x\,,\,y)\;=\;\tau_{min}\;+\;\left(\frac{N(x\,,\,y)\;+\;1}{2}\right)(\tau_{max}\;-\;\tau_{min})\;\in[\tau_{min}\,,\,\tau_{max}]$$

This ensures that Perlin noise is linearly transformed to fit in the desired resistance interval, producing a smooth cost function later on.

Terrain | Cost

```
SUBROUTINE GetTerrainResistance(x, y, minResistance, maxResistance)

    noise ← OctaveNoise(x, y, 3, 0.5, 2.0)
    normalised ← (noise + 1) / 2
    resistance ← minResistance + normalised * (maxResistance - minResistance)
    RETURN resistance

END SUBROUTINE


SUBROUTINE GetEdgeCost(nodeA, nodeB, world)

    midX ← (nodeA.x + nodeB.x) / 2
    midY ← (nodeA.y + nodeB.y) / 2
    resistance ← world.GetTerrainResistance(midX, midY)
    dx ← nodeA.x - nodeB.x
    dy ← nodeA.y - nodeB.y
    distance ← SQRT(dx*dx + dy*dy)
    RETURN resistance * distance

END SUBROUTINE
```

Where in the second subroutine the cost definition is implemented

$$C_{ij}\;=\;\tau_{ij}\;\cdot\;S_{ij}$$

Where $S_{ij}$is the Euclidean distance. This thereby Extends to

Energy Implementation | 2.3.4 |

This is an internal rule of an ant that dictates whether it has sufficient stamina to traverse from one end to the other. This is here to indicate increasing the plausibility of the biological archetype, by providing one of its core rules. This is also done to limit the entity from traversing an infinite distance away from the start position. This is also directly referenced in Beckers er al, where the ants have a set radius that it can travel to. In section 1.6.4 the energy is defined by the iterative formula

$$E_{ant}(t\;+\;1)=\;E_{ant}(t)\;-\;C_{ij}$$

This thereby prevents infinite traversal in the infinite terrain. Modelling, which can be considered to be defensive programming.

Energy

```
SUBROUTINE InitialiseAnt(ant, startNode, maxEnergy)

    ant.position ← startNode
    ant.startPosition ← startNode
    ant.maxEnergy ← maxEnergy
    ant.energy ← maxEnergy
    ant.exhausted ← False

END SUBROUTINE


SUBROUTINE HasSufficientEnergy(ant, edgeCost)

    RETURN ant.energy - edgeCost > 0

END SUBROUTINE


SUBROUTINE DeductEnergy(ant, edgeCost)

    ant.energy ← ant.energy - edgeCost
    IF ant.energy <= 0 THEN
        ant.exhausted ← True
        ant.energy ← 0
    END IF

END SUBROUTINE

SUBROUTINE ResetEnergy(ant)

    ant.energy ← ant.maxEnergy
    ant.exhausted ← False

END SUBROUTINE
```

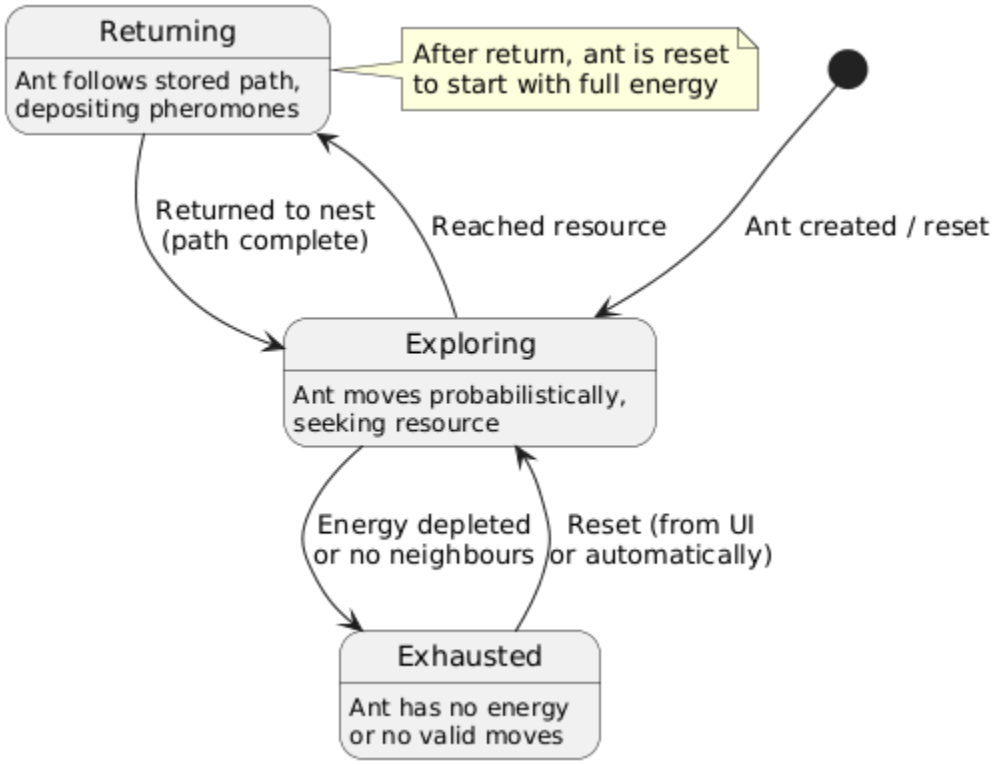
This pseudo code also provides a finite framework for the ant.

Movement | 2.3.5 |

Overview | 2.3.5.1 |

The movement algorithm is an assortment of multiple states that interact with each other in order to mimic stochastic movement, internal memory for returning, at what stage is the ant is to drop pheromones, and choosing logic.

This logic can be constructed as a state machine that shows a full journey of an ant and how it reacts to different scenarios. These will be deeply embedded into the ant behavior and therefore govern its movement logic which will cause emergent behavior that allows traversal through a dynamic environment that constantly changes. This can be demonstrated as follows:



Stochastic Movement | 2.3.5.2 |

In the analysis section 1.6.8 I modelled the movement to be as follows

$$P_{ij}(t) = \frac{\xi_{ij}(t) + \xi_0}{\sum_{k=1}^{n}(\xi_{ik}(t) + \xi_0)}$$

This is indeed a correct representation of probability but in classical ACO implementation, it is showed as

$$P_{ij}^{k} = \frac{[\tau_{ij}]^{\alpha} \cdot [\eta_{ij}]^{\beta}}{\Sigma_{l \in N^k}[\tau_{il}]^{\alpha} \cdot [\eta_{il}]^{\beta}}$$

Where alpha controls the influence of the trail τ_{ij} and beta controls the heuristic information η_{ij}. But in my implementation, I will not be having a secondary tuner for the heuristic because it can effectively make the governing rules more difficult to understand since the users of the application will then need a finite understanding of the equation. Therefore, simply set the beta value of the classical implementation to be equal to one as a conscious design choice. Therefore

$$P_{ij}(t) = \frac{\left(\xi_{ij}(t) + \xi_0\right)^{\alpha} \cdot \eta_{ij}}{\sum_{k=1}^{n}(\xi_{ik}(t) + \xi_0)^{\alpha} \cdot \eta_{ik}}$$

Where effectively the $\eta_{ij} = \frac{1}{C_{ij}}$ which encourages shorter path to be enhanced thereby formulating the full formula to be

$$P_{ij}(t) = \frac{\left(\xi_{ij}(t) + \xi_0\right)^{\alpha} \cdot \frac{1}{C_{ij}}}{\sum_{k=1}^{n}(\xi_{ik}(t) + \xi_0)^{\alpha} \cdot \frac{1}{C_{ij}}}$$

The division of cost appears in every term, therefore creating a bias towards the shorter path over the longer one.

Return Journey | 2.3.5.3 |

This state is initialized when an ant finds resources, as previously stated in Beckers et al. The ant only drops pheromone on the journey back only when they find food, otherwise no pheromone is dropped. Therefore, algorithmically, this creates a positive feedback loop that enhances the valid path to a pheromone. This in conjunction with the stochastic movement perfects it further by adding a trace of probability; this would cause the ant to diverge at some point and perhaps reach the resource in a shorter time. This in return increases the bias, and ant will tend to move towards it. And in accordance to the rule that governs decay

$$\xi_{ij}(t+1) = (1-\beta)\xi_{ij}(t) + \varepsilon\sigma_{ij}(t)$$

Because of this more ants will travel frequently in this path and therefore in consequence lay more pheromones along the way. This will be so that decay of the shorter path will affect less than that of the longer path due to the traversal frequency. This thereby allows emergent trial formations to be used to model an algorithm that allows to traverse a dynamic environment more efficiently and compare directly to the mathematically optimal path via the divergence metric defined in section 1.6.1

$$D_t = \frac{L_E(t) - L_O}{L_O}$$

Path Storage & Cycle removal | 2.3.5.4 |

During exploration, ants will keep an internal record of sequential order to allow it to traverse back to where it came from in case it finds a resource node. This path can be defined as a list of vertices that connects sequentially such that

$$path = [(v_0, v_1), (v_1, v_2), \ldots, (v_{n-1}, v_n)]$$

But due to the stochastic nature of ants, it will inevitably cause cycles to appear in the graphs, this must be removed so that ants won't be stuck in a ‘forever-loop’ that keeps repeatedly enhancing the current and the looped node which at some point will cause the accumulation of pheromone in the loop to exceed that of the emergent path that is meant to occur.

The process

This can be implemented by considering a simple graph of $A \rightarrow B \rightarrow C \rightarrow D \rightarrow B \rightarrow E$, In this graph a cycle can be identified from $A \rightarrow B$ onwards, so? Simply remove the $C \rightarrow D \rightarrow B$ and create a direct path such that $A \rightarrow B \rightarrow E$ which effectively eliminates my concern about the ‘forever-loop’. This can be implemented as follows

Removing Cycles

```
SUBROUTINE CleanPath(ant)

    seen ← EMPTY DICTIONARY
    cleanPath ← EMPTY LIST
    FOR EACH (a, b) IN ant.path DO

        IF a NOT IN seen THEN
            seen[a] ← LENGTH(cleanPath)
            cleanPath.ADD((a, b))

        ELSE

            cutIndex ← seen[a]
            cleanPath ← cleanPath[0:cutIndex]
            seen.CLEAR()
            FOR j FROM 0 TO LENGTH(cleanPath)-1 DO

                (x, y) ← cleanPath[j]
                seen[x] ← j

            END FOR

            cleanPath.ADD((a, b))
            seen[a] ← LENGTH(cleanPath)-1

        END IF
    END FOR

    ant.cleanPath ← cleanPath

END SUBROUTINE
```

Reversed Path | 2.3.5.5 |

The path of return is simply the reverse of the clean path in accordance to the edge notation of $(a\ ,\ b)$ where the corresponding return edge is $(b\ ,\ a)$. This is demonstrated by the pseudo code below

Return Path

```
SUBROUTINE BuildReturnPath(ant)

    ant.returnPath ← EMPTY LIST
    FOR i FROM LENGTH(ant.cleanPath)-1 DOWNTO 0 DO

        (a, b) ← ant.cleanPath[i]
        ant.returnPath.ADD((b, a))

    END FOR

    ant.returnIndex ← 0
    ant.returning ← True

END SUBROUTINE
```

Pheromone Deposition | 2.3.5.6 |

As the ant traverses back from the return path, which is initiated when the ant finds a resource node and then traverses back, It will need to drop pheromones in accordance to Beckers et al, Therefore model a pheromone update equation with base deposit strength ϑ and the length of the return path L , where the deposit amount is defined to be $\frac{\vartheta}{L}$ per edge. Thereby modelling

$$\xi_{ij}(t\ +\ 1)\ =\ \xi_{ij}\ +\ \frac{\vartheta}{L}$$

Where $\frac{\vartheta}{L}$ adds a natural bias towards the shorter path since pheromone is directly proportional to the probability of choosing a node and therefore an increasing increment means an increasing probability of the archetype to choose that edge. This further enhances the ant's internal rule by modifying the bias ratio in order for it to tend towards an optimal path. Over many iterations, the emergent path will get infinitely close to the mathematically optimal path. This can be shown idealistically as

$$\lim_{t\to\infty} D_t\ =\ \lim_{t\to\infty}\ \ \frac{L_E(t)\ -\ L_O}{L_O}\ =\ 0$$

This divergence is a direct consequence of the positive feedback loop between the probability, decay, and pheromone bias. This naturally is a very idealistic scenario, since there will be a small transient probability that will shift the emergent path ever so slightly be a set standard deviation such that

$\lim_{t\to\infty} D_t = \lim_{t\to\infty} \frac{L_E(t) - L_O}{L_O} \approx 0 + \delta$, and therefore, being modelled in the following

Update Rule

```
SUBROUTINE ReturnStep(ant, pheromones, depositStrength, world)

    IF ant.returnIndex >= LENGTH(ant.returnPath) THEN

        ant.hasDeposited ← True
        RETURN

    END IF

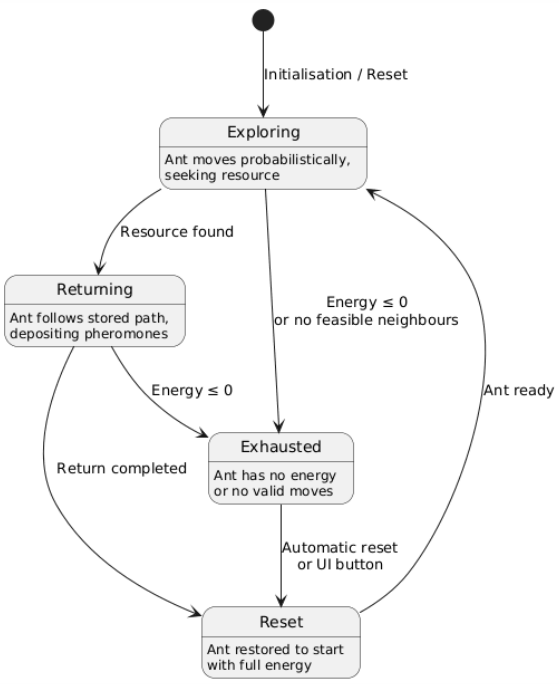
    edge ← ant.returnPath[ant.returnIndex]
    normEdge ← (MIN(edge.from, edge.to), MAX(edge.from, edge.to))
    pathLength ← LENGTH(ant.returnPath)
    depositAmount ← depositStrength / pathLength

    pheromones[normEdge] ← pheromones.get(normEdge, 0) + depositAmount
    ant.returnCost ← ant.returnCost + world.getEdgeCost(edge.from, edge.to)
    ant.position ← edge.to
    ant.returnIndex ← ant.returnIndex + 1

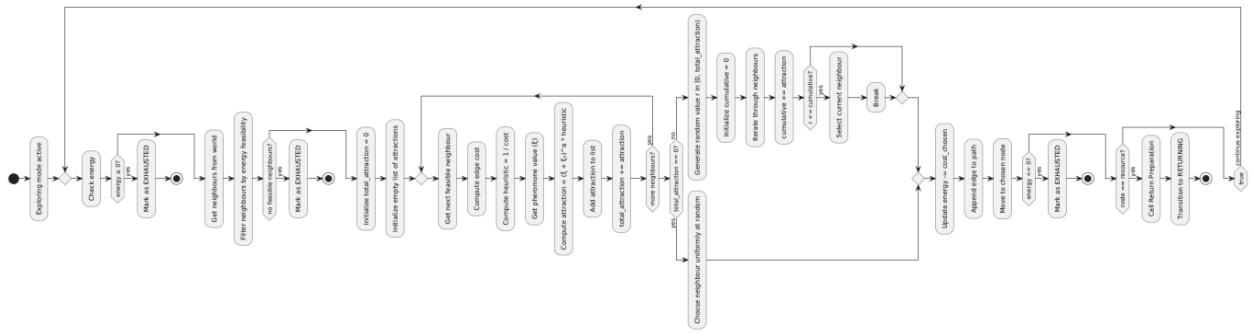
END SUBROUTINE
```

Summary for Movement | 2.3.5.7 |

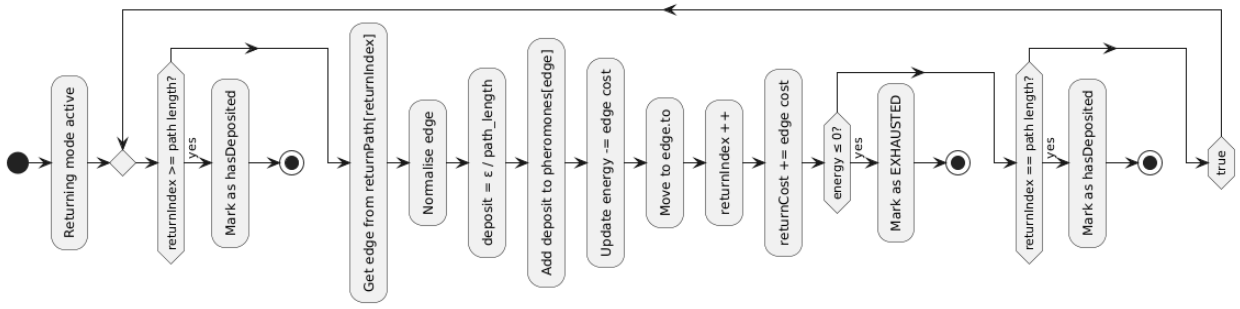
Ants Life, Summary of how an moves in a foreign environment



Explorative movement Action Diagram



Return State Action Diagram



These diagrams show a finite expression for the two major components of an ant; this is a vertical image pasted as a horizontal image; Download it as a pdf then flip it 90 degrees and zoom in then it's readable or read from the attached files.

Emergent Path Trace | 2.3.6 |

The emergent path represents the current path that is formed to be the best route in accordance with the pheromone trial that is produced by the entity. This algorithm aims to do a greedy walk from the start node to the resource tracing through the pheromone strength.

Process

Starting from the start node, repeatedly check neighbors and check for the strongest pheromone trail. This process starts after the first ant moves and shows nothing if there's no path. Then on the second iteration, ignoring the node that the trace came from (otherwise it will cause cycles), check the neighbors with the strongest pheromone, and the cumulative cost must not exceed the ant energy. If neighbors have equal weight, then the next node will be chosen probabilistically.

The length $L_E(t)$ is the total cost of the traced path

$$L_E(t) = \sum_{(i,j) \in Edge} C_{ij}$$

Where the cost is summed across the accessible neighbors.

Emergent Path

```
FUNCTION TraceEmergentPath(start, target, world, pheromones, maxEnergy)
    current ← start
    visited ← EMPTY SET
    pathEdges ← EMPTY LIST
    totalCost ← 0
    reached ← False

    FOR step ← 1 TO 1000 DO
        IF current = target THEN
            reached ← True
            BREAK
        END IF

        visited.ADD(current)
        neighbours ← world.getNeighbours(current)
        bestNode ← None
        bestStrength ← -1

        FOR EACH n IN neighbours DO
            IF n IN visited THEN
                CONTINUE
            END IF

            edge ← (MIN(current, n), MAX(current, n))
            strength ← pheromones.get(edge, 0)

            IF strength > bestStrength THEN
                bestStrength ← strength
                bestNode ← n
            END IF
        END FOR

        IF bestNode IS None THEN
            BREAK
        END IF

        cost ← world.getEdgeCost(current, bestNode)
        totalCost ← totalCost + cost
        IF totalCost > maxEnergy THEN
            BREAK
        END IF

        pathEdges.ADD((current, bestNode))
        current ← bestNode
    END FOR

    RETURN pathEdges, reached, totalCost
END FUNCTION
```

A-Star Path | 2.3.7 |

A-Star will have a global perspective of the world and therefore have a pre-defined set of nodes that are allowed to be traversed or not. This will allow it to traverse the graph and find the optimal path for every iteration.

Process

Traverse through the pre-defined grid using the Euclidean distance, this will be the heuristic for the path algorithm

$$h(n) = \mu \cdot dist(n, goal)$$

Where μ is the estimated minimum terrain resistance.

Where the edge cost will be the same as the one for the emergent path where it is defined as C_{ij}

Therefore, defining the predicted cost to be as follows

$$f(n) = C_{ij} + \mu \cdot dist(n, goal)$$

Helper

```
SUBROUTINE Heuristic(node, target)

    RETURN ABS(node.x - target.x) + ABS(node.y - target.y)

END SUBROUTINE

SUBROUTINE ReconstructPath(cameFrom, current)

    path ← [current]
    WHILE current IN cameFrom DO
        current ← cameFrom[current]
        path.PREPEND(current)
    END WHILE
    RETURN path

END SUBROUTINE
```

These two methods will be used in conjunction with the main body of A-star. These functions simply calculate the Euclidean distance and return the path of A star. This is defined in the following abstraction

AStar

```
SUBROUTINE AStar(start, target, world)
    openSet ← PRIORITY_QUEUE()
    cameFrom ← EMPTY DICTIONARY
    gScore ← EMPTY DICTIONARY
    fScore ← EMPTY DICTIONARY

    gScore[start] ← 0
    fScore[start] ← Heuristic(start, target)
    openSet.ENQUEUE(start, fScore[start])

    WHILE NOT openSet.IS_EMPTY() DO
        current ← openSet.DEQUEUE()

        IF current = target THEN
            RETURN ReconstructPath(cameFrom, current)

        END IF

        FOR EACH neighbour IN world.getNeighbours(current) DO
            tentativeG ← gScore[current] + world.getEdgeCost(current, neighbour)

            IF tentativeG < gScore.get(neighbour, INFINITY) THEN
                cameFrom[neighbour] ← current
                gScore[neighbour] ← tentativeG
                fScore[neighbour] ← gScore[neighbour] + Heuristic(neighbour, target)

                IF neighbour NOT IN openSet THEN
                    openSet.ENQUEUE(neighbour, fScore[neighbour])

                END IF
            END IF
        END FOR
    END WHILE

    RETURN None

END SUBROUTINE
```

Visualization & Animation ~ User Experience | 2.3.8 |

Overview | 2.3.8.1 |

This section will design the needs of the users by creating visual artefacts and atmospheric choices to make the user experience better.

Draw | 2.3.8.2 |

Draw Path | 2.3.8.2.1 |

To implement the path algorithms in the above section, namely the emergent path and the A star path, where the emergent will be drawn as cyan and optimal will be drawn as a magenta line. this will be drawn in the high-level interface as follows

Drawing Path

```
SUBROUTINE DrawPath(screen, pathEdges, colour, worldOffset)
    FOR EACH (fromNode, toNode) IN pathEdges DO
        screenFrom ← (fromNode.x - worldOffset.x, fromNode.y - worldOffset.y)
        screenTo   ← (toNode.x - worldOffset.x, toNode.y - worldOffset.y)
        DrawLine(screen, colour, screenFrom, screenTo, width=number)
    END FOR
END SUBROUTINE
```

Draw Entities | 2.3.8.2.2 |

To draw the obstacles that must be present in the high-level interface, it will be done as follows

Drawing Obstacles

```
CLASS Obstacle
    constructor(worldX, worldY):
        worldPos ← (worldX, worldY)
        visualSeed ← hash(worldX, worldY)
        random ← RANDOM(visualSeed)

        offsetX ← random(-MAX_OFFSET, MAX_OFFSET)
        offsetY ← random(-MAX_OFFSET, MAX_OFFSET)

    draw(screen, worldOffset):

        screenX ← worldPos.x - worldOffset.x + offsetX
        screenY ← worldPos.y - worldOffset.y + offsetY
        DrawObstacleShape(screen, screenX, screenY, visualParams)
END CLASS
```

Animation | 2.3.8.2.3 |

This section directly aims to capture a pleasant atmosphere that beautifies the interface at the background. This thereby allows me to introduce idle entities that does not really affect the

implementation of the functionality but simply aims to enhance user experience. This will be implemented by the below abstraction

Idle Animation

```
CLASS AnimatedObject

    constructor(baseX, baseY):
        basePos ← (baseX, baseY)
        angle ← random(0, 2π)
        speed ← random(minSpeed, maxSpeed)
        radius ← random(minRadius, maxRadius)
        phase ← random(0, 2π)

    update(dt):
        angle ← angle + speed * dt

    draw(screen, worldOffset):
        dx ← cos(angle) * radius
        dy ← sin(angle) * radius
        screenX ← basePos.x - worldOffset.x + dx
        screenY ← basePos.y - worldOffset.y + dy

END CLASS
```

This will be used in two areas

1. Fireflies – made to oscillate about a certain point - Cosmetic.
2. Tree Animation – light canopy made to oscillate about a point – Cosmetic.

Data Structures | 2.3.9 |

Structure	Purpose	Justification
Pheromone dictionary	Store pheromone in the designated vertex	O(1) access + stored as sorted tuples to prevent duplicates
World nodes (set)	Walkable Nodes	Prevents duplicates + fast access
Blocked nodes (set)	Obstacle positions	O(1) checks
Ant path (list)	Internal record of the return path	Ordered path to be able to trace back
Priority queue	A* open set	Efficiently get lowest f-cost node

Complexity Analysis | 2.3.10 |

Component	Operation	Time Complexity	Notes
Chunk Loading	Generate r-radius neighborhood	$O(r^2)$	r is fixed (small constant)
Chunk Creation	Initialise chunk dictionary	$O(1)$ per chunk	Only occurs when new chunk entered
Node Generation	Populate nodes in chunk	$O(s)$	s = nodes per chunk (constant)
Perlin Sampling	Single noise evaluation	$O(o)$	o = number of octaves (constant)
Terrain Resistance	Noise + scaling	$O(o)$	Depends on octave count
Edge Cost Calculation	Distance + resistance	$O(1)$	Constant arithmetic
Ant Movement	Evaluate neighbors	$O(d)$	d = max degree (≤ 4)
A* Algorithm	Shortest path search	$O(E \log V)$	Using priority queue
Emergent Path Trace	Greedy traversal	$O(L)$	L = path length

Parameter Justification | 2.3.11 |

Parameter	Default Value	Mathematical Role	Justification	Behavior if Increased
β (Evaporation Rate)	0.0009	Controls decay	$t_{\frac{1}{2}} = \frac{\ln 2}{\beta} = 770$	Weaker reinforcement
ε (Deposit Strength)	5.0	Scales pheromone deposition	A set amount that is placed by ant when returning	Stronger bias
ξ_0 (Baseline Pheromone)	0.1	Prevents zero-probability transitions	Prevents 0 division error	Reduces impact on pheromone deposited
α (Pheromone Bias)	2.0	Exponent controlling trail sensitivity	Controls probability	Reduces random probability
Ant Count	50	Number of parallel agents	Sample range	Faster convergence
Max Energy	5000	Energy constraint	Upper bound ensures finite exploration of an ant	Able to reach resource destination
Steps per Frame	10	Simulation speed multiplier	A balanced frame rate	Faster convergence

Symbol Definition | 2.3.12 |

Symbol	Meaning
$\Omega \in \mathbb{R}^2$	Continuous spatial domain
C_{ij}	Cost of traversing edge between nodes i and j
S_{ij}	Euclidean distance between nodes i and j
$\tau(x, y)$	Terrain resistance at coordinate (x,y)
(τ_{min}, τ_{max})	Minimum and maximum terrain resistance
$N(x, y)$	Perlin noise function
$(\xi_{\{ij\}(t)})$	Pheromone intensity on edge (i,j) at time t
ξ_0	Baseline pheromone constant
α	Pheromone influence exponent
β	Pheromone evaporation rate
ε	Deposit strength constant
$(\sigma_{\{ij\}(t)})$	Number of ants depositing on edge (i,j) at time t
$(P_{\{ij\}(t)})$	Probability of selecting neighbor j from node i
$(\eta_{\{ij\}})$	Heuristic value $\frac{1}{C_{\{ij\}}}$
$L_{E(t)}$	Emergent path length at time t
L_O	Optimal path length (A*)
D_t	Divergence metric
n_a	Number of ants